

OSAC Storage: LVMS as a CSI Backend — Options Analysis

Working document for the Storage WG · prepared for discussion · reflects merged code + in-flight PRs + the OSAC-2872 design as of review.

TL;DR. The OSAC Storage Control Plane (OSAC-2872) was designed exclusively for **network-attached** arrays (VAST/NetApp/Pure). Its entire model — provision a Volume via the API independently of any pod, vendor controllers on the hub, credential isolation — assumes the storage is reachable from anywhere. **LVMS is node-local and sticky**: a volume physically lives in a volume group on one node and can only be created once the consuming pod's node is known. LVMS therefore **cannot** fit the design cleanly. The three options below are three different **levels of compromise**, not three equivalent implementations. None makes LVMS behave like a network tier. Crucially, **LVMS was introduced for a dev-only purpose** — giving developers usable storage without a real backend array. That premise (not a "who is the user?" question) should drive the decision: it removes any hard requirement for tenant-facing inventory, quota, or a uniform provisioner, and points away from the heaviest option.

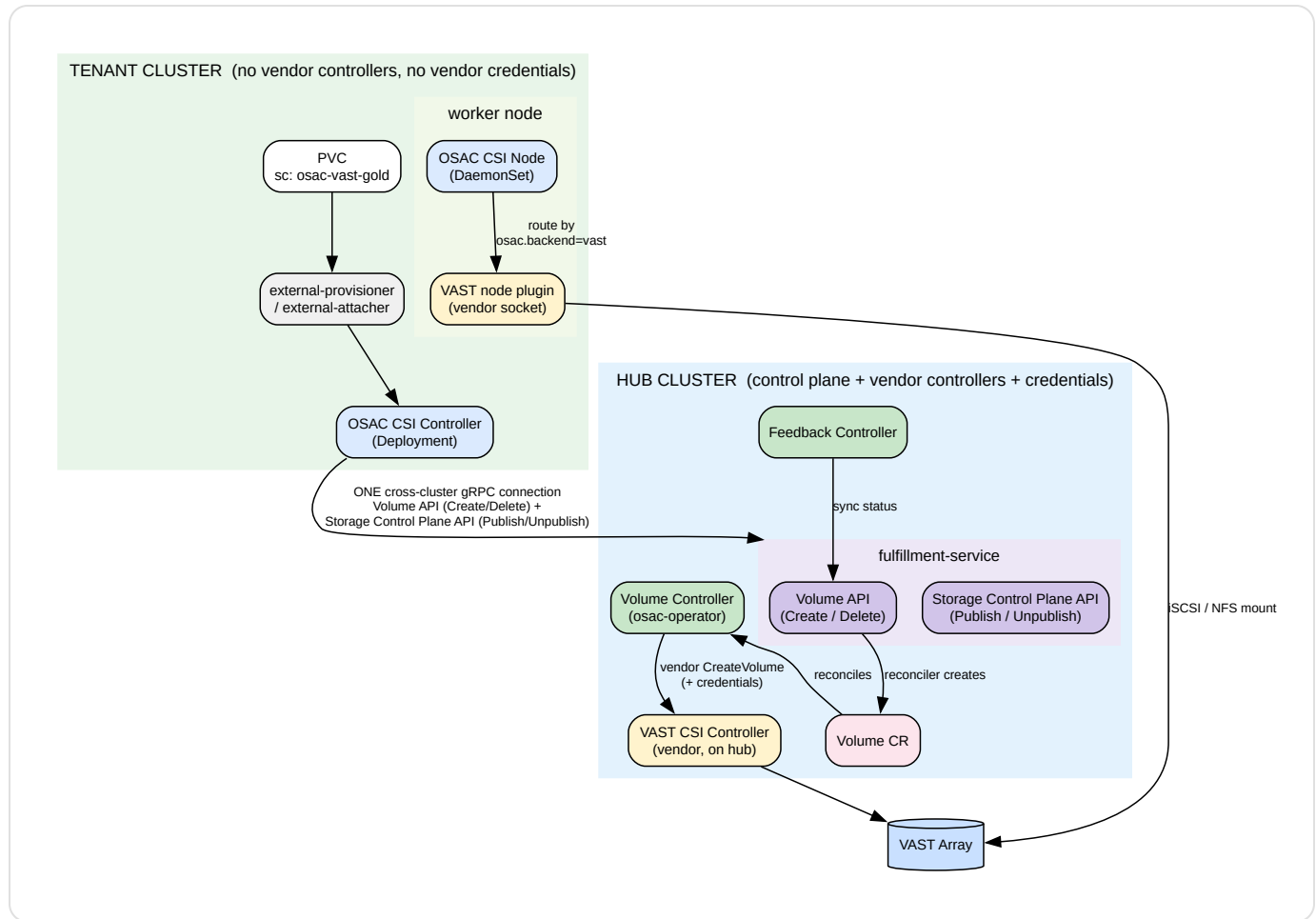
1. Background: how the OSAC Storage Control Plane works (network-attached)

Four repos plus the installer make up the storage control plane. The key design principles, quoted from the OSAC-2872 design:

- **Asynchronous, declarative Volume API.** The CSI driver calls `CreateVolume`, which persists a record and returns immediately. A reconciler then creates a `Volume` CR on the hub; the operator drives the vendor call; a feedback controller syncs status back.
- **"The tenant cluster has no vendor controllers and no vendor credentials."** All controller operations route through the fulfillment-service. Vendor **controllers** run on the **hub**; only vendor **node plugins** run on the tenant cluster.
- **"The CSI driver never connects directly to vendor controllers."** The CSI **node plugin** is a proxy (it routes mount calls to a vendor node socket by

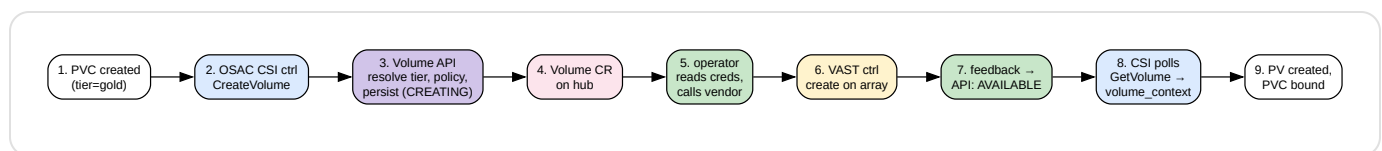
`volume_context["osac.backend"]`). The CSI **controller** is *not* a proxy — it talks only to the fulfillment-service.

DEPLOYMENT TOPOLOGY (VAST)



Note where things live: vendor **controller** + credentials + operator on the **hub**; vendor **node plugin** on the tenant worker. The OSAC CSI driver has exactly one cross-cluster connection — to the fulfillment-service — and never to a vendor controller.

2. Reference flow: VAST (network-attached) provisioning



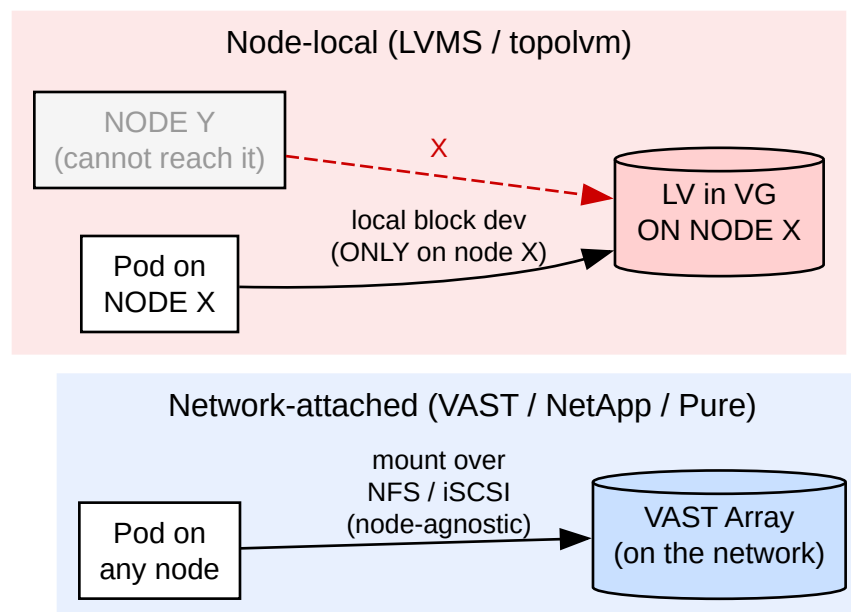
This is the **CSI / PVC-driven** path (dynamic provisioning). The other entry is **API-first**: the Volume is created via the API with no consumer, then a pre-provisioned PV (`csi.volumeHandle = <osac volume id>`) is bound statically and `CreateVolume` is never called. API-first is out of scope here — it is impossible for LVMS (see §3.1) — so only the CSI-driven path is elaborated.

Numbered, matching the OSAC-2872 sequence:

1. Tenant creates PVC (storageClassName: osac-<vendor>-gold)
2. external-provisioner → OSAC CSI Controller.CreateVolume(tier=gold, tenant, size,
3. Volume API: resolve tier→backend+protocol, OPA policy check, persist record (sta
4. fulfillment reconciler (on pg_notify) creates a Volume CR on the HUB
5. operator Volume controller: reads tenant creds from hub Secret, calls vendor CSI
6. VAST CSI controller creates the volume on the array, returns vendor volume ID
7. feedback controller → Volume API: Update(state=AVAILABLE, vendor_volume_id) + Si
8. OSAC CSI polls GetVolume until AVAILABLE → returns volume_context{osac.backend,
9. Kubernetes creates the PV (volume_context stored as spec.csi.volumeAttributes),

Mount is a separate timeline, triggered not by provisioning but by **a consumer being scheduled onto a node** — a pod, or for VMaaS the KubeVirt VM's `virt-launcher` pod. It repeats every time a consumer starts on a node; provisioning happens only once. Sequence: pod/VM scheduled → `VolumeAttachment` created → external-attacher → `ControllerPublishVolume` → Storage Control Plane API → operator → vendor `ControllerPublishVolume` (attach); then kubelet → OSAC CSI **node** → `NodeStageVolume` / `NodePublishVolume`, routed by `osac.backend=vast` → VAST node plugin → iSCSI/NFS mount. **No control-plane call is made from the node**; all routing is baked into `volume_context` at create time.

3. Why LVMS does not fit: the node-local constraint



VAST is node-agnostic: the array is on the network, any node mounts it. LVMS is the opposite — the volume is an LV carved from a volume group **on one specific node**, and it is only reachable from that node. Consequences that ripple through the whole architecture:

- **WaitForFirstConsumer is mandatory.** You cannot pick where to put the volume until the scheduler picks the pod's node.
- **Topology must flow end-to-end.** Scheduler picks node X → CSI `CreateVolume` receives `accessibility_requirements.preferred=[{topology.topology.io/node: X}]` → provisioning must target X → the response's `accessible_topology` pins the PV to X.
- **Node affinity must be propagated to the PV.** The **LV** (the LVM block device) is physically on node X — that's a given. The challenge is making Kubernetes aware of it: the **PV** must carry `spec.nodeAffinity=node X` so the scheduler only places consumers there. A VAST PV has no node affinity (mountable anywhere); an LVMS PV must be pinned to node X for its lifetime, and that pin has to be threaded through the provisioning path.

3.2 Deployment asymmetry: why the vendor-controller-on-hub model breaks for LVMS

A CSI driver is two halves — a **controller** (Deployment) and a **node plugin** (DaemonSet). OSAC splits them deliberately:

Piece	Where it runs	Why
Vendor controller (VAST/NetApp/Pure)	Hub	Credential isolation — vendor creds stay on the hub, never reach the tenant cluster. The controller just makes a network API call to the array , so it doesn't matter where nodes are.
Vendor node plugin	Tenant / CaaS	Must run where the mounts happen; reached by the OSAC node plugin via socket routing.
OSAC CSI driver (controller + node)	Tenant / CaaS (and the hub too, per the passthrough proposal)	That's where tenant PVCs are. Its only cross-cluster reach is one gRPC connection to the fulfillment-service.

topolvm-controller cannot follow the vendor-controller-on-hub pattern. Unlike VAST's controller (which makes a network call to a remote array), topolvm-controller on `CreateVolume` creates a `LogicalVolume` CR in its own cluster's API, which the **topolvm-node DaemonSet in that same cluster** watches and acts on, and its scheduling reads **that cluster's node VG capacity**. It operates entirely within one cluster and can only manage that cluster's own nodes. Run it on the hub and it would target hub nodes, not tenant nodes. So it **must** live on the tenant

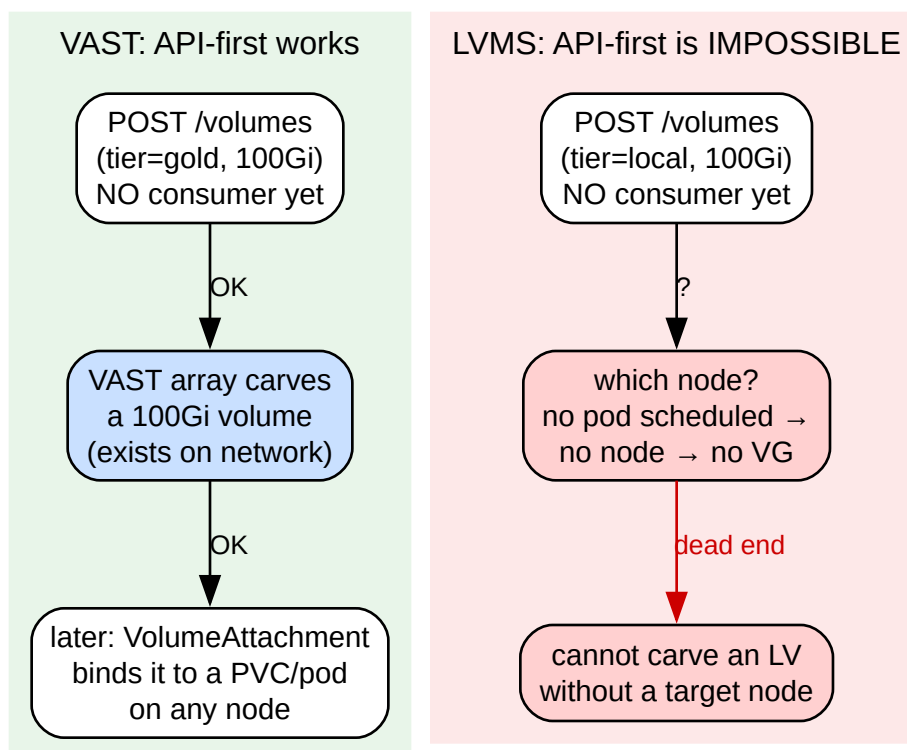
cluster — exactly where OSAC-3011/3234 already install LVMS. This is the concrete reason Option A's "operator on the hub calls the vendor controller" model does not translate to LVMS.

3.3 How a CSI controller is normally reached (context for Option B)

A CSI controller pod contains the vendor plugin container **plus** the CSI sidecars (external-provisioner, external-attacher, external-resizer). The plugin serves its CSI gRPC over a **unix-domain socket on an in-pod `emptyDir`**, and the **only clients are those co-located sidecars**. external-provisioner watches PVCs → calls `CreateVolume` on the socket; external-attacher watches VolumeAttachments → calls `ControllerPublish`. Nothing outside the pod ever talks to topolvm-controller; it is never on the network. That is why any option where a *different* pod (the OSAC CSI controller) drives topolvm-controller needs a new access path — see Option B.

3.1 The deepest mismatch: API-first provisioning

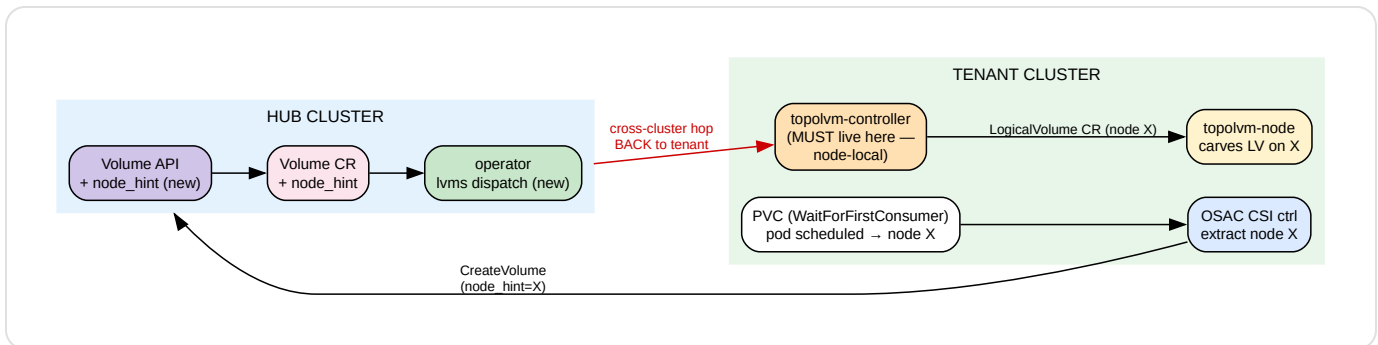
With `pvc_ref` now removed from the Volume API (PR #341, merged) and a future **VolumeAttachment** object (OSAC-3278) taking over the Volume ↔ PVC linkage, the OSAC Volume lifecycle is **decoupled from pods**. The canonical path becomes **API-first**: create a Volume with no consumer, attach it later. This is natural for a network array — and **structurally impossible for LVMS**.



This is the single strongest architectural argument: the direction the Volume API is evolving (provision-in-advance, pod-independent) is exactly the direction LVMS cannot follow. For LVMS the **only** viable entry point is the CSI + `WaitForFirstConsumer` path, where a node exists at provision time. The pure-API path must be forbidden for local tiers.

4. The three options

Option A Full chain — CSI → fulfillment → Volume CR → operator → topolvm



COMPONENTS

OSAC CSI controller (+ topology extraction), Volume API (+ a new `node_hint` /topology field), Volume CR (+ `node_hint`), operator Volume controller (+ an lvms `VendorProvisioner`), topolvm on the tenant cluster, OSAC CSI node (+ topology labels).

DATA FLOW

1. Pod scheduled → node X (`WaitForFirstConsumer`)
2. OSAC CSI ctrl extracts node X from `accessibility_requirements`
3. → `fulfillment.CreateVolume(tier=local, size, accessMode, node_hint=X)` [Volume
4. tier resolution → backend=lvms; DB record; Volume CR on HUB (spec: ..., `node_hir`
5. operator lvms dispatch → creates a LogicalVolume CR targeting node X ON THE TENA
6. operator polls LV ready → Volume CR status (AVAILABLE); feedback → DB
7. OSAC CSI polls `GetVolume` → `volume_context` + `accessible_topology=[node X]`
8. node plugin routes `osac.backend=lvms` → `topolvm-node` socket → mount

Two structural problems. (1) **Control-flow round-trip:** the request climbs from the tenant cluster to the hub (fulfillment + CR + operator), then the operator must reach **back down into the tenant cluster** to create the LogicalVolume on node X — for a purely local operation the hub has no part in. (2) **topolvm-controller cannot live on the hub** where VAST's controller lives; it must run on the tenant cluster with the nodes and VGs. So step 5

is either the operator **reimplementing topolvm-controller's job** (fragile, duplicative) or the operator **delegating to topolvm-controller** — in which case **Option A = Option B plus fulfillment/CR/ operator bookkeeping and a cross-cluster hop.**

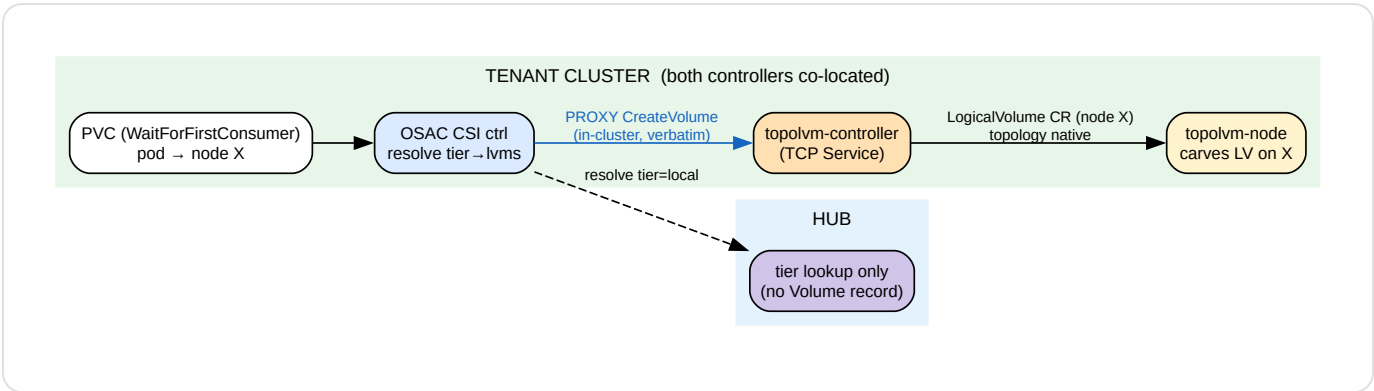
PROS

- ✓ Full volume inventory in the OSAC DB
- ✓ Quota enforcement possible
- ✓ One provisioner name (`osac-csi`) — looks uniform to the tenant

CONS

- ✗ Needs a new topology field in the Volume API (Volume API owner / OSAC-3273/3280) — not there today
- ✗ Needs an operator lvms provisioner — not there today
- ✗ Cross-cluster reach hub → tenant to create the LV
- ✗ **Still** cannot support API-first / provision-in-advance (§3.1) — no VAST parity despite the most work
- ✗ Most engineering of the three

Option B Passthrough — CSI controller forwards local provisioning to topolvm



TWO REALIZATIONS (CONFIRM WHICH WITH THE PROPOSER — THEY DIFFER IN WHAT MUST BE EXPOSED)

	B1 · gRPC proxy	B2 · CRD create
OSAC CSI ctrl does	Proxies the CSI <code>CreateVolume</code> RPC to topolvm-controller	Directly creates a topolvm <code>LogicalVolume</code> CR

Requires	topolvm-controller exposed via a TCP Service (non-standard)	A k8s client + RBAC in the CSI driver (none today)
Security surface	Network-exposes a normally in-pod socket	RBAC to create one CRD type
Note	Bypasses topolvm's own provisioner sidecar	Reimplements the thin part of topolvm-controller (CR creation)

Sidecar nuance: with StorageClass `provisioner: osac-csi`, **OSAC's** external-provisioner sidecar handles the PVC and topolvm's own provisioner sidecar sits idle (it only watches `provisioner: topolvm.io`). So Option B effectively **replaces topolvm's provisioner sidecar with a cross-pod hop** (B1) or with direct CR creation (B2).

COMPONENTS

OSAC CSI controller (+ an lvms proxy branch), fulfillment (tier-resolution lookup only, no Volume record), topolvm-controller exposed via a TCP Service, OSAC CSI node (existing socket routing).

DATA FLOW

1. Pod scheduled → node X
2. OSAC CSI ctrl: resolve tier=local → backend=lvms + topolvm-controller endpoint
3. OSAC CSI ctrl PROXIES CreateVolume verbatim → topolvm-controller (in-cluster)
4. topolvm-controller creates the LogicalVolume on node X, returns AccessibleTopolvm
5. OSAC CSI returns topolvm's response, adding volume_context{osac.backend=lvms}
6. node plugin routes osac.backend=lvms → topolvm-node socket → mount

This is the design's rejected Alternative 3 ("CSI driver proxies vendor calls / client-side orchestration"). It was rejected for **credential isolation** — vendor credentials would transit the tenant cluster. **But LVMS has no vendor credentials** (local disk, no array API, no Secret), so the specific reason for the rejection **does not apply to LVMS**. That makes B a defensible exception rather than a violation.

Co-location detail: on both shapes the OSAC CSI controller and topolvm-controller are on the **same** cluster (CaaS: both on the guest cluster per OSAC-3234; VMaaS/hub: both on the hub per OSAC-3011), so the path is an **in-cluster** hop — strictly simpler than A's cross-cluster reach. The one new mechanic applies to **B1 only**: topolvm-controller isn't network-exposed by default (its csi-provisioner sidecar uses an in-pod unix socket), so a TCP Service must be added — pre-condition to confirm with the CSI driver owner / topolvm. **B2** sidesteps this entirely (create the `LogicalVolume` CR directly; needs only RBAC).

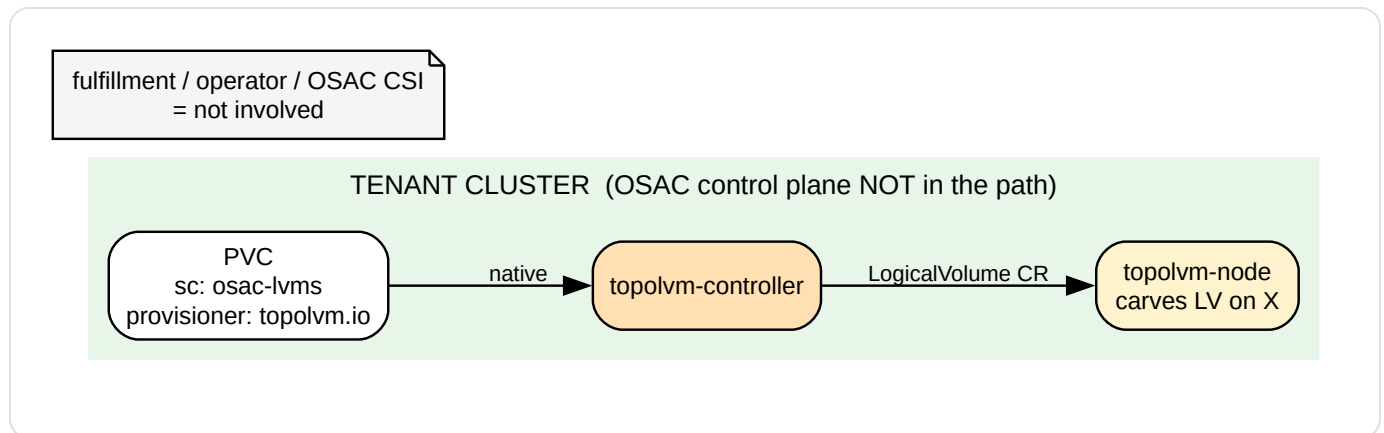
PROS

- ✓ No dependency on the operator lvms path or a new Volume-API field
- ✓ Topology "just works" — topolvm owns it natively
- ✓ Low latency (direct, in-cluster); provisioner name stays `osac-csi`
- ✓ The credential-isolation objection is moot for LVMS

CONS

- ✗ No Volume record → no inventory, no quota for LVMS
- ✗ The controller behaves differently per backend (proxy for lvms, full-chain for network) — a split path to maintain
- ✗ New exposure: TCP Service for topolvm-controller (B1) or a CR-creation RBAC client in the CSI driver (B2)

Option C Native — keep the topolvm/AAP path, OSAC not in the LVMS path



COMPONENTS

A topolvm StorageClass created by AAP (already shipped via OSAC-3011/3234), `provisioner: topolvm.io`. The OSAC CSI driver, fulfillment, and operator are not involved in the LVMS path at all.

DATA FLOW

```
PVC (sc=osac-lvms, WaitForFirstConsumer) → topolvm-controller → LogicalVolume → topolvm-node (zero OSAC control-plane involvement)
```

PROS

- ✓ Zero new work — already functional and merged
- ✓ Correct by construction (topolvm as designed); no new security surface, no cross-cluster hops
- ✓ Honest about LVMS being a different category

CONS

- ✗ StorageClass provisioner is `topolvm.io`, not `osac-csi` — an **abstraction leak** (tenant sees LVMS internals)
- ✗ No OSAC inventory or quota for LVMS
- ✗ Two visibly different tenant UXes (local tier vs network tiers)

5. Side-by-side comparison

Every row where a "network parity" property is ✗ is a compromise that **no option removes** — because they stem from LVMS being node-local, not from implementation choices.

Dimension	VAST (reference)	A · Full chain	B · Passthrough	C · Native
OSAC volume inventory	✓	✓	✗	✗
Quota enforcement	✓	✓	✗	✗
Provisioner = <code>osac-csi</code>	✓	✓	✓	✗ (<code>topolvm.io</code>)
API-first / provision-in-advance	✓	✗	✗	✗
Topology handled by	n/a	OSAC (4 layers)	topolvm	topolvm
Cross-cluster reach needed	✓ (by design)	✗ hub → tenant	in-cluster only	none
New security surface	—	operator → tenant	topolvm TCP Service	none
Follows OSAC-2872 design	✓	partly	rejected Alt. 3*	rejected Alt. 4

New engineering	—	Highest	Medium	~None
-----------------	---	---------	--------	-------

* Alternative 3 was rejected for credential isolation, which does not apply to LVMS.

6. Open questions for the WG

Settled premise: LVMS is dev-only (usable storage without a real backend array). So there is no tenant-facing requirement for inventory, quota, or a uniform provisioner — which removes the only justification for Option A. The remaining questions are about B vs C.

1. **Does the tenant-visible provisioner name matter even for dev use?** (`osac-csi` vs `topolvm.io`) — if no, C is sufficient; if we want dev flows to exercise the real `osac-csi` path, B.
2. **Do we want dev flows to exercise the OSAC CSI path at all** (so dev testing mirrors the network-backend path), or is native `topolvm` fine for dev?
3. **[B pre-condition]** Which realization — B1 (expose `topolvm-controller` via a TCP Service) or B2 (CSI driver creates the `LogicalVolume` CR directly, RBAC only)? Confirm with the CSI driver owner / `topolvm`.
4. **Are we OK maintaining a split controller path (B)** long term — passthrough for `lvms`, full-chain for network backends?

Option A is retained below for completeness only; the dev-only premise rules it out.

7. Recommendation

Given the dev-only premise, it's B vs C. Option C is already done and correct — if native `topolvm` (provisioner `topolvm.io`) is acceptable for dev use, stop there. **Option B** is worth it only if we want dev flows to exercise the real `osac-csi` path (so dev testing mirrors the network-backend experience) — topology is correct for free and the credential objection is moot for LVMS; the split path is the only real cost. Prefer the **B2** realization (direct `LogicalVolume` CR creation, RBAC only) over B1 (TCP-exposing `topolvm-controller`). **Option A is ruled out** by the dev-only premise: it buys tenant-facing inventory/quota nobody needs here, and still cannot achieve VAST parity (API-first is impossible for LVMS regardless) — A-delegate is just B plus bookkeeping and a cross-cluster hop, A-reimplement duplicates `topolvm-controller`.

Appendix A — Verified current code state

Area	State (verified)
VolumeSpec proto	<code>storage_tier</code> , <code>size_gib</code> , <code>access_mode</code> only. <code>pvc_ref</code> removed (PR #341 merged). No topology/ <code>node_hint</code> field.
Tier resolution	Merged (PR #342): Volume Create resolves tier → backend+protocol, writes <code>status.backend</code> / <code>status.protocol</code> .
OSAC CSI controller	Delegates to fulfillment VolumeClient (PR #141 merged). No topology extraction, no <code>AccessibleTopology</code> in response, no <code>VOLUME_ACCESSIBILITY_CONSTRAINTS</code> capability.
OSAC CSI node	Routes mount to a vendor socket by <code>osac.backend</code> (proxy.Manager). <code>NodeGetInfo</code> returns only <code>node_id</code> — no topology labels.
operator Volume controller	PR #340 (open): <code>VendorProvisioner</code> interface, currently nil. No <code>lvms/topolvm/LogicalVolume</code> awareness anywhere in the codebase.
Volume reconciler (fulfillment)	PR #339 (open).
Vendor CSI controllers deploy	PR #188 (open): csi-backends Helm chart (VAST/Trident/Pure) — network backends only.

Appendix B — What OSAC-3702 must add on the CSI side (any of A/B)

- `ControllerGetCapabilities` += `VOLUME_ACCESSIBILITY_CONSTRAINTS` (required for `WaitForFirstConsumer`)
- `CreateVolume` : extract `accessibility_requirements.preferred[0]`, return `AccessibleTopology`
- `NodeGetInfo` : return `topology.topolvm.io/node=<nodeID>`
- Helm: mount the `topolvm-node` socket into the OSAC CSI node container; register `lvms=<socket>` in `--vendor-sockets`

Sources: OSAC-2872 storage-control-plane design.md (authoritative), verified against merged code (PR #141/#341/#342) and open PRs (#339/#340/#188), OSAC-1110 StorageTier design (single-backend-per-tier), OSAC-3011/3234 (LVMS install). Diagrams rendered locally with graphviz.